



EXO-STELLAR NEXUS

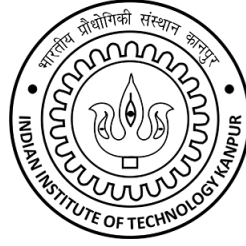
Summer Project 2024



Exo-Stellar Nexus

Astronomy Club, IITK

Summer Project '24



Acknowledgements

We are deeply grateful to our mentors for their invaluable guidance and support. We would also like to thank the Astronomy Club, IITK, and the Science and Technology Council, IITK, for providing us with this wonderful opportunity.

Mentors:

- Ayush SunilRao Bokad → bokadayush22@iitk.ac.in
- Sanskaar Srivastava → sanskaars22@iitk.ac.in
- Ananthula Abhivardhan → ananthulan22@iitk.ac.in

Mentees:

- Rishabh Verma → rishabv23@iitk.ac.in
- Aryan Kumar → aryank23@iitk.ac.in
- Shohom De → shohomde23@iitk.ac.in
- Toshika Kamble → toshikak23@iitk.ac.in

Contents

1	Introduction: Charting the Cosmos	3
1.1	Project Overview	3
1.2	Motivation	3
1.3	Efforts	3
2	Python and its libraries	4
2.1	NumPy: Numerical Python	4
2.2	Pandas: Data Manipulation Made Easy	4
2.3	Matplotlib: Plotting the Universe	5
2.4	OpenCV: Towards a new vision	5
3	Image processing techniques and HAAR Cascades	6
3.1	Image Processing	6
3.2	HAAR Cascades	8
4	Classification Algorithms	10
4.1	Logistic Regression	10
4.2	Support Vector Machine (SVM)	10
4.3	Decision Tree	11
4.4	Random Forest	11
4.5	K-Nearest Neighbors (KNN)	12
4.6	Naive Bayes	13
4.7	Linear Discriminant Analysis (LDA)	13
5	Constellations and constellation detector model	14
5.1	Constellation detector model	14
6	Exoplanets, detection methods, habitability parameters	22
6.1	Detection methods	22
6.2	Planetary Habitability: The Potential for Life Beyond Earth	25
6.3	Finding Potential Habitable Planets from Transit Data	26
7	Conclusion and Future Work	27
7.1	Habitability Prediction Model	27
7.2	Constellation Detection Model	29

1 Introduction: Charting the Cosmos

1.1 Project Overview

The project aims to cover a wide domain of astronomy and its real world application of machine learning. Project started from very basic and touched advanced tech aspects for detection of constellations in night sky and detection of habitable exoplanets for future human endeavors.

Project has covered astronomy, observational astronomy, and application of machine learning modeling to achieve its final goal. Mapping the cosmos for constellation detection and learning contribution to the journey of Interstellar quest for survival by detecting habitable exoplanets serves our two main objectives of the project.

1.2 Motivation

The objectives of the project are motivated by the aim to find another habitable planet and if there is life and nature as we see it somewhere else other than the Earth. It is a difficult task and this project will add on to the global avenues going on in this field.

As for the constellation detector model, it is very difficult to know by just looking at the night sky without prior knowledge about the location from which it is being viewed, about the stars and constellations that are visible. Some people are interested but are unable to identify the stars by solely looking at them, this model aims at helping them by just feeding it an image and giving them the names of the stars and constellations visible in the photograph.

1.3 Efforts

Previously there have been research papers written and models created by Julia-Wang and the pair of Kartikeya Gupta and Anikait Sahota. Their research papers tell us the thinking and methodology as to how they approached the problem. The model by Kartikeya is functioning however the accuracy is not that high.

There is an entire section of NASA by the name SEEC that is dedicated to searching for exoplanets and their habitability, the Kepler mission, K2 all fall under it. The James Webb Space Telescope, again a part of it, launched in 2021, is designed to detect minute differences in wavelengths to determine what chemical elements are present in exoplanetary atmospheres.

2 Python and its libraries

Python is an easy to use, general purpose programming language. Its syntax is similar to English - which makes it easy to read and write. Python programming language (latest Python 3) is being used in web development, and Machine Learning applications, along with all cutting-edge technology in the Software Industry.

We are using python because it has many great libraries for data handling and machine learning models that we can use in our projects. It has several libraries - developed for particular tasks. For example, NumPy is a Python library known for its high-level mathematical functions and speed. For handling astronomical data in this project, we used NumPy, Pandas, Matplotlib, OpenCV and Scikit.

2.1 NumPy: Numerical Python

NumPy facilitates working with large arrays and multidimensional matrices. It offers a plethora of mathematical functions, random number generators, linear algebra tools, and more. In our code snippets, we refer to NumPy as np.

Here are some of the things that numpy can do.

- NumPy has special functions to create diagonal, identity or zero matrices.
- NumPy has several useful mathematical functions like Trigonometric, Hyperbolic, Logarithmic etc. It also has useful tools like Convolutions of 1D sequences and the Fast Fourier transform
- NumPy arrays follow zero based indexing. This means that the indexing starts at 0 and ends at n-1 for nth element of an array
- Addition(+) and Subtraction(-) arithmetic operations are defined over arrays. These follow element wise addition/subtraction.

Here are some of the code snippets :-

```
1 import numpy as np
2 # Creating an identity matrix
3 identity_matrix = np.eye(4)
4 print(identity_matrix)
```

Listing 1: NumPy Example

2.2 Pandas: Data Manipulation Made Easy

Pandas stands for "Python Data Analysis Library". This library simplifies the data pre-processing steps and facilitates working with data. A key feature of Pandas is that it converts data in files such as csv, excel sheets etc into a tabular data frame. This makes our life much much simpler. Here are some of the things we can do with Pandas.

- Data Visualisation.
- Dataset cleaning, merging and joining.
- Easy handling of missing data in floating point as well as non-floating point data.

The following code reads a csv file and stores it into a data frame named "data".

```
1 import pandas as pd
2 # Creating a DataFrame
3 info = {'Name': ['Star1', 'Star2'], 'Magnitude': [1.5, 2.3]}
4 data = pd.DataFrame(data)
5 print(data)
```

Listing 2: Pandas Example

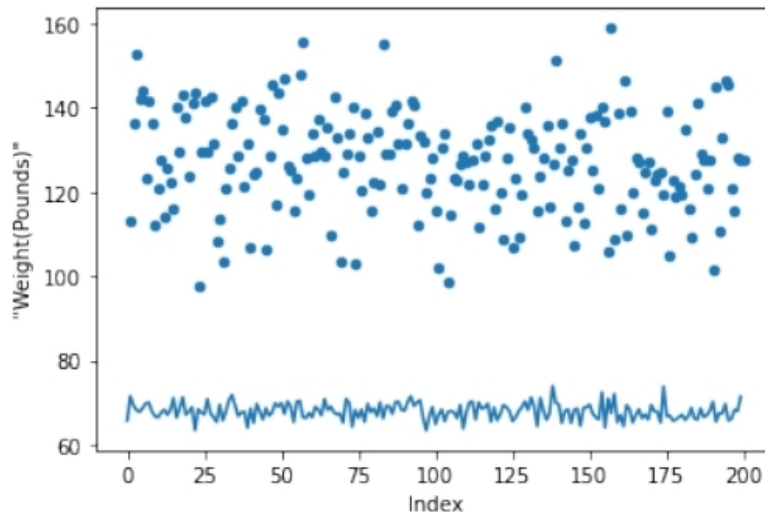
2.3 Matplotlib: Plotting the Universe

This library is responsible for plotting numerical data. And that's why it is used in data analysis. It is also an open-source library and plots high-defined figures like pie charts, histograms, scatter-plots, graphs, etc. It makes easy things easy and hard things possible. Matplotlib offers a plethora of options that we can use on our plots to tweak and adjust the appearance of our plots.

Remember the dataset from data.csv that we created in the example of pandas library. Now we are going to use matplotlib to make a plot of index vs weight and a scatter plot of height on the same graph.

```
1 import matplotlib.pyplot as plt
2 data.plot.scatter(x=data.columns[0], y=data.columns[2])
3 data[data.columns[1]].plot()
```

Listing 3: Matplotlib Example



Plot obtained from the data

2.4 OpenCV: Towards a new vision

OpenCV, short for Open Source Computer Vision Library, is an Open-source computer vision and machine learning software library. It plays a major role in real-time operation which is very important in today's systems.

By using it, one can process images and videos to identify objects, faces, or even the handwriting of a human. We can use many image processing techniques to manipulate the image we

are concerned with. In our project we are using it to process images of the night sky to make them clearer for constellation detection.

```
1
2 import cv2
3 import numpy as np
4 img=cv2.imread("image1.jpeg")
5 img2=img.copy()
6 img2[:, :, 0]=0
7 img2[:, :, 1]=0
8 stack=np.hstack((img, img2))
9 cv2.imwrite("processed.jpeg", stack)
10 cv2.imshow("cat", stack)
11 cv2.waitKey(0)
```

Listing 4: OpenCV Example

3 Image processing techniques and HAAR Cascades

3.1 Image Processing

Digital Image processing is the class of methods that deal with manipulating digital images through the use of computer algorithms. It is an essential pre-processing step in many applications, such as face recognition, object detection, and image compression. Image processing is done to enhance an existing image or to sift out important information from it. This is important in several Deep Learning-based Computer Vision applications, where such pre-processing can dramatically boost the performance of a model. Manipulating images, for example, adding or removing objects to images, is another application, especially in the entertainment industry.

There are many types of techniques that we can use to manipulate images. In our project we are using some of those techniques to extract crucial information. Here are some of the image processing techniques that we are using in our project.

- (a) **Image Enhancement:-** One of the most common image processing tasks is an image enhancement, or improving the quality of an image. Image enhancement techniques can be applied manually using image editing software, or automatically using algorithms and computer programs such as OpenCV. It consists of many techniques like filtering, contrast-brightness adjustment, Denoising, etc.

Some techniques of image enhancement that we are using in the project are described briefly below:-

- 1 **Sharpening:-** Sharpening is the process of enhancing the edges and fine details in an image to make it appear sharper and more defined. It is important because it can help to bring out the details and features in an image, making it more visually appealing and easier to understand. Sharpening can be used to correct blur or softness in an image and can be applied using a variety of techniques.
- 2 **Contrast-Brightness Adjustment:-** Adjusting the brightness and contrast of an image can significantly affect its visual appeal and effectiveness. It can also help to correct defects or flaws in the image and make it easier to see details. Finding the

right balance of brightness and contrast is important for creating an attractive and effective image.

There are several ways to adjust the brightness and contrast of an image using OpenCV and Python. One common method is to use the `cv2.addWeighted()` function, which allows you to adjust the brightness by adding a scalar value to each pixel in the image, and the contrast by scaling the pixel values.

- 3 **Filtering:-** Filtering is the most fundamental operation in computer vision and image processing. It is the manipulation of pixel values based on the filtering function applied at that location. The value for the pixel is calculated by applying a kernel and then the central value is replaced by the calculated value. Some of the known filtering methods in OpenCV are Gaussian Filtering, Median Filtering, etc.

- (b) **Image Segmentation:-** Image segmentation is a computer vision technique that partitions a digital image into discrete groups of pixel—image segments—to inform object detection and related tasks. By parsing an image's complex visual data into specifically shaped segments, image segmentation enables faster, more advanced image processing.

Some techniques of image enhancement that we are using in the project are described briefly below:-

- 1 **Thresholding:-** Threshold-based segmentation is one of the simplest and most straightforward image segmentation techniques. It involves converting a grayscale image into a binary image by applying a threshold value. Pixels with intensity values above the threshold are classified into one category, while those below the threshold are classified into another.
- 2 **Edge Detection:-** Edge-based segmentation focuses on identifying the boundaries between different regions in an image. This technique detects significant changes in intensity or color, which typically indicate the presence of edges.

- (c) **Object Recognition and Detection:-** Object Detection is very important for Computer vision domains, this concept (Object Detection) identifies and locates objects in images or videos. Object detection finds extensive applications across various sectors.

Object recognition is the technique of identifying the object present in images and videos. It is one of the most important applications of machine learning and deep learning. The goal of this field is to teach machines to understand (recognize) the content of an image just like humans do.

These are the main techniques that are being used in this project. Some other image processing techniques are as follows:-

- **Image Restoration:-**

This step deals with improving the appearance of an image and is an objective operation since the degradation of an image can be attributed to a mathematical or probabilistic model. For example, removing noise or blur from images.

- **Color Image Processing:-**

This step aims at handling the processing of colored images (16-bit RGB or RGBA images), for example, performing color correction or color modeling in images.

- **Wavelets and Multi-Resolution Processing :-**

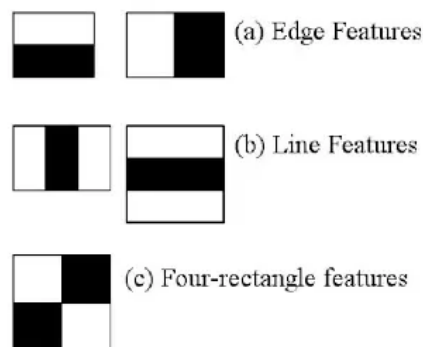
Wavelets are the building blocks for representing images in various degrees of resolution. Images subdivision successively into smaller regions for data compression and for pyramidal representation.

- **Image Compression:-**

For transferring images to other devices or due to computational storage constraints, images need to be compressed and cannot be kept at their original size. This is also important in displaying images over the internet; for example, on Google, a small thumbnail of an image is a highly compressed version of the original. Only when you click on the image is it shown in the original resolution. This process saves bandwidth on the servers.

3.2 HAAR Cascades

Haar cascade is an algorithm that can detect objects in images, irrespective of their scale in image and location. This algorithm is not so complex and can run in real-time. We can train a haar-cascade detector to detect various objects like cars, bikes, buildings, fruits, etc. Haar cascade uses the cascading window, and it tries to compute features in every window and classify whether it could be an object.



HAAR Cascades as Classifiers

Haar cascade works as a classifier. It classifies positive data points → that are part of our detected object and negative data points → that don't contain our object.

- Haar cascades are fast and can work well in real-time.
- Haar cascade is not as accurate as modern object detection techniques are.
- Haar cascade has a downside. It predicts many false positives.
- Simple to implement, less computing power required.

HAAR Features:

- HAAR features are simple rectangular patterns used to detect edges, lines, or regions within an image.
- Each feature consists of light and dark regions, which help in identifying contrasts in the image.

Combining HAAR Features:

- Individual HAAR features are weak classifiers. They are combined to form a strong classifier capable of distinguishing between positive (e.g., faces) and negative (e.g., non-faces) images.
- This combination process is typically handled by algorithms such as AdaBoost.

Pixel Value Calculation:

- **Equation:**

- The equation

$$V[i, j] = \sum \sum I[m - i, n - j] H[i, j] \quad (1)$$

is used to calculate the value of a pixel by summing contributions from surrounding pixels using the HAAR feature. However, this is computationally expensive.

Integral Image Concept

- **Integral Image:**

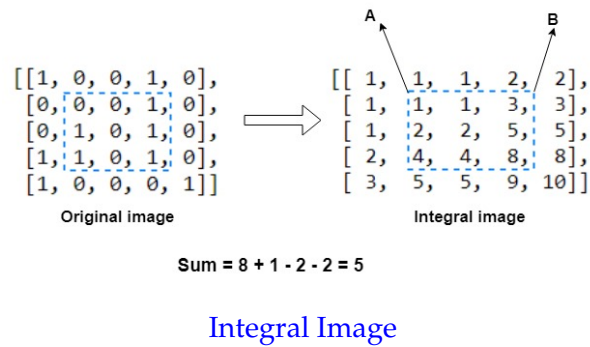
- An integral image, also known as a summed-area table, allows for the rapid calculation of the sum of pixel values within a rectangular region.
 - Each element at position (x, y) in the integral image represents the sum of all pixel values above and to the left of (x, y) in the original image.

- **Efficiency:**

- Using the integral image, any rectangular sum can be computed in constant time, significantly speeding up the processing time for HAAR feature evaluation.

The OpenCV library manages a repository containing all popular haar cascades that can be used for:

- Human face detection
- Eye detection
- Nose / Mouth detection
- Vehicle detection



4 Classification Algorithms

Predicting whether an exoplanet can support life involves classifying it as "potentially habitable" or "not habitable" based on features from transit data. Transit data provides critical information about exoplanets, such as their size, orbit, and host star characteristics. Here's a brief overview of some key binary classification algorithms used for this purpose.

4.1 Logistic Regression

Logistic regression is a statistical model used for binary classification. It predicts the probability of an exoplanet being habitable based on input features.

1. Mechanism:

- Uses the logistic function to estimate habitability probability.
- Outputs a score between 0 and 1, with a threshold (usually 0.5) for classification.

2. Applications:

- Initial screening of potentially habitable exoplanets.
- Works well with linear relationships.

3. Pros:

- Simple and interpretable.
- Low computational requirements.
- Provides probabilistic outputs.

4. Cons:

- Limited to linear relationships.
- Can struggle with imbalanced datasets.

4.2 Support Vector Machine (SVM)

SVM aims to find the optimal hyperplane that separates habitable and non-habitable exoplanets with maximum margin.

1. Mechanism:

- Transforms data into a higher-dimensional space using kernel functions.
- Maximizes the margin between classes.

2. Applications:

- Suitable for complex, non-linear relationships.
- Effective for high-dimensional datasets.

3. Pros:

- Effective in high-dimensional spaces.
- Robust to overfitting.

4. Cons:

- Computationally intensive.
- Requires careful tuning.

4.3 Decision Tree

A Decision Tree uses a tree-like structure to make decisions based on feature values, splitting data into subsets to predict habitability.

1. Mechanism:

- Splits data into branches based on feature values that maximize information gain.
- Each node represents a feature test; each leaf node represents a class label.

2. Applications:

- Visualizing decision-making processes.
- Suitable for non-linear relationships.

3. Pros:

- Easy to interpret and visualize.
- Handles both numerical and categorical data.

4. Cons:

- Prone to overfitting.
- Can be unstable with small data changes.

4.4 Random Forest

Random Forest is an ensemble method that builds multiple decision trees and aggregates their predictions for more accuracy.

1. Mechanism:

- Trains each tree on a random data subset.
- Aggregates predictions from all trees.

2. Applications:

- Robust to overfitting.
- Effective for large, complex datasets.

3. Pros:

- Handles large datasets and high-dimensional spaces.
- Provides feature importance scores.

4. Cons:

- Computationally intensive.
- Less interpretable than individual trees.

4.5 K-Nearest Neighbors (KNN)

KNN classifies exoplanets based on the majority class among the k-nearest neighbors.

1. Mechanism:

- Measures distance between data points and assigns the majority class among neighbors.

2. Applications:

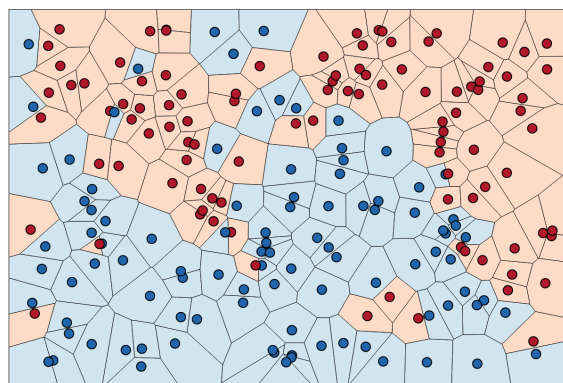
- Useful for small datasets.
- Effective for detecting local patterns.

3. Pros:

- Simple and intuitive.
- No training phase required.

4. Cons:

- Computationally expensive during prediction.
- Performance degrades with high-dimensional data.



Boundaries in K-NN model

4.6 Naive Bayes

Naive Bayes is a probabilistic classifier based on Bayes' theorem, assuming feature independence.

1. Mechanism:

- Calculates the probability of each class given the input features.
- Selects the class with the highest probability.

2. Applications:

- Adaptable for numerical data related to exoplanet characteristics.

3. Pros:

- Works well with high-dimensional data.
- Fast and efficient.

4. Cons:

- Assumes feature independence.
- Can be less accurate than complex models.

4.7 Linear Discriminant Analysis (LDA)

LDA projects data onto a lower-dimensional space and uses a linear decision boundary to separate classes.

1. Mechanism:

- Maximizes the distance between class means while minimizing within-class variance.

2. Applications:

- Effective for linear separability.
- Useful for dimensionality reduction.

3. Pros:

- Effective for linear separability.
- Also aids in dimensionality reduction.

4. Cons:

- Assumes normal distribution of features.
- Struggles with non-linear relationships.



5 Constellations and constellation detector model

Constellations are groups of stars in the night sky that form a recognizable pattern. These patterns can range from something like a crown to a lion. These patterns are named after animals, mythological figures and objects long ago. Most of the constellations have some story associated with their name. There are 88 constellations that have been recognized by the IAU. A map of 88 constellations projected into either the equatorial or galactic coordinates. Out of 88 constellations 56 of them are present in the north hemisphere and the rest 32 at the southern hemisphere of the globe. As earth moves around the sun we see different patches of night sky at different times of a year. That's why we observe only a few constellations at a certain night of the year. Zodiac constellations are the 12 constellations that lie along the ecliptic plane. There is a perception that the sun passes through the 12 constellations throughout the year.

5.1 Constellation detector model

Training and Validation:

- Split the dataset into training, validation, and test sets to ensure robust model evaluation.
- Train the model using labeled images where the positions of stars are annotated.
- Validate and fine-tune the model to optimize its performance.

Evaluation:

- Evaluate the model's accuracy, precision, recall, and F1 score using the test dataset.
- Compare the model's performance with existing star detection methods to assess improvements.

Methodology:

1. Data Collection:

- Source images from astronomical databases.

- Use image augmentation techniques to increase the diversity of the training data.

2. Model Architecture:

- Implement a CNN architecture with layers optimized for feature extraction from images.
- Experiment with different configurations of convolutional, pooling, and dense layers to find the best-performing model.

3. Training Process:

- Use labeled data to train the model, applying techniques like early stopping and learning rate scheduling to enhance training efficiency.
- Perform cross-validation to ensure the model generalizes well to unseen data.

4. Training Process:

- Use confusion matrices to analyze true positives, false positives, true negatives, and false negatives.
- Calculate metrics such as accuracy, precision, recall, and F1 score to quantify model performance.

```

1 import cv2
2 import numpy as np
3 from matplotlib import pyplot as plt
4 import math
5 import os
6 import copy
7 from sklearn.neighbors import KNeighborsClassifier
8 from sklearn.model_selection import train_test_split
9 from google.colab import drive
10
11 # Mount Google Drive
12 drive.mount('/content/drive')
13
14 # Define paths
15 root_path = '/content/drive/My Drive'
16 template_directory = os.path.join(root_path, 'constellations')
17 image_directory = os.path.join(root_path, 'images')
18
19 def dist(p1, p2):
20     return math.sqrt((p1[0] - p2[0]) ** 2 + (p1[1] - p2[1]) ** 2)
21
22 def getAngle(p0, p1, p2):
23     return math.acos((dist(p0, p1) ** 2 + dist(p0, p2) ** 2 - dist(p1, p2) **
24         2) / (2 * (dist(p0, p1) ** 2) * (dist(p0, p2) ** 2)))
25
26 def getNormalisedCoordinates(x, y, brightest_index, second_brightest_index,
27     lines=[]):
28     x = copy.deepcopy(x)
29     y = copy.deepcopy(y)
30
31     lines = np.array(lines)
32     for line in lines:
33         for x1, y1, x2, y2 in line:
34             x1 -= x[brightest_index]
35             y1 -= y[brightest_index]
36             x2 -= x[brightest_index]
37             y2 -= y[brightest_index]
38             line[0][0] = x1

```

```

37         line[0][1] = y1
38         line[0][2] = x2
39         line[0][3] = y2
40
41     for i in range(len(x)):
42         x[len(x) - i - 1] -= x[brightest_index]
43         y[len(x) - i - 1] -= y[brightest_index]
44
45     distance_brightest = math.sqrt((x[brightest_index] - x[
second_brightest_index]) ** 2 + (y[brightest_index] - y[
second_brightest_index]) ** 2)
46
47     for i in range(len(x)):
48         x[i] = x[i] / distance_brightest
49         y[i] = y[i] / distance_brightest
50     lines = lines / distance_brightest
51
52     p0 = (x[brightest_index], y[brightest_index])
53     p1 = (x[second_brightest_index], y[second_brightest_index])
54     p2 = (1, 0)
55     theta = getAngle(p0, p1, p2)
56     if round(x[second_brightest_index] * math.cos(theta) - y[
second_brightest_index] * math.sin(theta), 2) != float(1) or round(x[
second_brightest_index] * math.sin(theta) + y[second_brightest_index] * math
.cos(theta), 2) != float(0):
57         theta = -theta
58
59     for i in range(1, len(x)):
60         x_new = x[i] * math.cos(theta) - y[i] * math.sin(theta)
61         y_new = x[i] * math.sin(theta) + y[i] * math.cos(theta)
62         x[i], y[i] = round(x_new, 2), round(y_new, 2)
63
64     for line in lines:
65         for x1, y1, x2, y2 in line:
66             x1_new = x1 * math.cos(theta) - y1 * math.sin(theta)
67             y1_new = x1 * math.sin(theta) + y1 * math.cos(theta)
68             x2_new = x2 * math.cos(theta) - y2 * math.sin(theta)
69             y2_new = x2 * math.sin(theta) + y2 * math.cos(theta)
70             line[0][0] = x1_new
71             line[0][1] = y1_new
72             line[0][2] = x2_new
73             line[0][3] = y2_new
74
75     return np.array(x), np.array(y), lines
76
77 def iterateArea(contours, lines=[], iterate=False):
78     lines = np.array(lines)
79
80     coordinates = {}
81     for i in range(len(contours)):
82         cnt = contours[i]
83         M = cv2.moments(cnt)
84         cx = int(M['m10'] / M['m00'])
85         cy = int(M['m01'] / M['m00'])
86         area = cv2.contourArea(cnt)
87         if area in coordinates:
88             coordinates[area + 0.0001] = (cx, cy)
89         else:
90             coordinates[area] = (cx, cy)
91
92     sorted_area = []
93     for i in reversed(sorted(coordinates.keys())):
94         sorted_area.append(i)

```

```

95
96     x = []
97     y = []
98     for area in sorted_area:
99         x.append(coordinates[area][0])
100        y.append(coordinates[area][1])
101    if len(sorted_area) < 3:
102        return [], [], []
103
104    threshold = min(150, sorted_area[2])
105    count = 0
106    for area in sorted_area:
107        if area >= threshold:
108            count += 1
109
110    coordinates_list = []
111    if not iterate:
112        return getNormalisedCoordinates(x, y, 0, 1, lines)
113    else:
114        for i in range(count):
115            for j in range(i + 1, count):
116                x_new, y_new, _ = getNormalisedCoordinates(x[i:], y[i:], i, j,
117                lines)
118                coordinates_list.append((x_new, y_new))
119
120        return coordinates_list
121
122    def findEdges(image, thresh1, thresh2):
123        out = cv2.Canny(np.array(image), thresh1, thresh2)
124        return out
125
126    def invertImage(image):
127        return cv2.bitwise_not(image)
128
129    def getGrayscale(image):
130        return cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
131
132    def applyMedian(image, size):
133        return cv2.medianBlur(image, size)
134
135    def binariseImage(img, thresholds):
136        output_thresh = []
137        for threshold in thresholds:
138            ret, thresh = cv2.threshold(img, threshold, 255, cv2.THRESH_BINARY)
139            output_thresh.append(thresh)
140        return output_thresh
141
142    def getGreenChannel(img):
143        image_copy_green = img.copy()
144        image_copy_green[:, :, 0] = 0
145        image_copy_green[:, :, 2] = 0
146        return image_copy_green
147
148    def getBlueChannel(img):
149        image_copy_blue = img.copy()
150        image_copy_blue[:, :, 1] = 0
151        image_copy_blue[:, :, 2] = 0
152        return image_copy_blue
153
154    def getRedChannel(img):
155        image_copy = img.copy()
156        image_copy[:, :, 1] = 0
157        image_copy[:, :, 0] = 0
158        return image_copy

```

```

157
158 def makeTemplates(template_directory):
159     templates_coordinates = {}
160
161     for filename in os.listdir(template_directory):
162         img = cv2.imread(os.path.join(template_directory, filename))
163         red_channel = getRedChannel(img)
164         thresh = binariseImage(red_channel, [165.75, 191.25])
165
166         blue_channel = getBlueChannel(img)
167         thresh2 = binariseImage(red_channel, [125.75, 255])
168         letters = thresh2[0] - thresh2[1]
169         new_blue = blue_channel + letters
170
171         for i in range(len(new_blue)):
172             for j in range(len(new_blue[i])):
173                 if new_blue[i][j][2] != 0:
174                     new_blue[i][j][1] = 0
175                     new_blue[i][j][0] = 0
176                     new_blue[i][j][2] = 0
177
178         final = thresh[0] - thresh[1]
179         stars = applyMedian(final, 3)
180         lines = applyMedian(new_blue, 3)
181         stars_grey = getGrayscale(stars)
182         lines_grey = getGrayscale(lines)
183         final_stars = binariseImage(stars_grey, [20])
184         final_lines = binariseImage(lines_grey, [5])
185         final_stars_inverted = invertImage(final_stars[0])
186         final_lines_inverted = invertImage(final_lines[0])
187         final_lines = applyMedian(final_lines[0], 3)
188
189         edged = findEdges(final_stars_inverted, 30, 200)
190
191         rho = 1
192         theta = np.pi / 180
193         threshold = 10
194         min_line_length = 2
195         max_line_gap = 3
196         line_image = np.copy(img) * 0
197
198         drawn_lines = cv2.HoughLinesP(final_lines, rho, theta, threshold, np.
199 array([], min_line_length, max_line_gap)
200
201         if drawn_lines is not None:
202             for line in drawn_lines:
203                 for x1, y1, x2, y2 in line:
204                     cv2.line(line_image, (x1, y1), (x2, y2), (255, 0, 0), 5)
205
206         edge_copy = edged.copy()
207         contours, hierarchy = cv2.findContours(edge_copy, cv2.RETR_EXTERNAL,
208 cv2.CHAIN_APPROX_NONE)
209         final_contours = [contour for contour in contours if cv2.contourArea(
210 contour) != 0]
211
212         x, y, normalised_lines = iterateArea(final_contours, drawn_lines)
213         if len(x) == 0 or len(y) == 0:
214             continue
215
216         templates_coordinates[filename.split('.')[0]] = (x, y, len(x),
217 normalised_lines)
218
219     return templates_coordinates

```

```

216
217 def featureExtraction(template):
218     x, y, num_stars, lines = template
219     feature_vector = np.zeros(num_stars * 2 + len(lines.flatten()))
220     feature_vector[:num_stars] = x
221     feature_vector[num_stars:num_stars*2] = y
222     feature_vector[num_stars*2:] = lines.flatten()
223     return feature_vector
224
225 def prepare_data(templates_coordinates):
226     X = []
227     y = []
228     labels = list(templates_coordinates.keys())
229     label_to_index = {label: index for index, label in enumerate(labels)}
230
231     max_length = max(len(featureExtraction(template)) for template in
232 templates_coordinates.values())
233
234     for label, template in templates_coordinates.items():
235         feature_vector = featureExtraction(template)
236         padded_feature_vector = np.pad(feature_vector, (0, max_length - len(
237 feature_vector)), 'constant')
238         X.append(padded_feature_vector)
239         y.append(label_to_index[label])
240
241     X = np.array(X)
242     y = np.array(y)
243     return X, y, labels
244
245 # Prepare data
246 templates_coordinates = makeTemplates(template_directory)
247 X, y, labels = prepare_data(templates_coordinates)
248
249 # Split the data into training and testing sets
250 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
251 random_state=42)
252
253 # Train KNN classifier
254 knn = KNeighborsClassifier(n_neighbors=3)
255 knn.fit(X_train, y_train)
256
257 # Evaluate the classifier
258 accuracy = knn.score(X_test, y_test)
259 print(f"Model accuracy: {accuracy * 100:.2f}%")
260
261 def classify_image(image_path):
262     img = cv2.imread(image_path)
263     red_channel = getRedChannel(img)
264     thresh = binariseImage(red_channel, [165.75, 191.25])
265
266     blue_channel = getBlueChannel(img)
267     thresh2 = binariseImage(red_channel, [125.75, 255])
268     letters = thresh2[0] - thresh2[1]
269     new_blue = blue_channel + letters
270
271     for i in range(len(new_blue)):
272         for j in range(len(new_blue[i])):
273             if new_blue[i][j][2] != 0:
274                 new_blue[i][j][1] = 0
275                 new_blue[i][j][0] = 0
276                 new_blue[i][j][2] = 0
277
278     final = thresh[0] - thresh[1]

```

```

276 stars = applyMedian(final, 3)
277 lines = applyMedian(new_blue, 3)
278 stars_grey = getGrayscale(stars)
279 lines_grey = getGrayscale(lines)
280 final_stars = binariseImage(stars_grey, [20])
281 final_lines = binariseImage(lines_grey, [5])
282 final_stars_inverted = invertImage(final_stars[0])
283 final_lines_inverted = invertImage(final_lines[0])
284 final_lines = applyMedian(final_lines[0], 3)
285
286 edged = findEdges(final_stars_inverted, 30, 200)
287
288 rho = 1
289 theta = np.pi / 180
290 threshold = 10
291 min_line_length = 2
292 max_line_gap = 3
293 line_image = np.copy(img) * 0
294
295 drawn_lines = cv2.HoughLinesP(final_lines, rho, theta, threshold, np.array
([], min_line_length, max_line_gap)
296
297 if drawn_lines is not None:
298     for line in drawn_lines:
299         for x1, y1, x2, y2 in line:
300             cv2.line(line_image, (x1, y1), (x2, y2), (255, 0, 0), 5)
301
302 edge_copy = edged.copy()
303 contours, hierarchy = cv2.findContours(edge_copy, cv2.RETR_EXTERNAL, cv2.
CHAIN_APPROX_NONE)
304 final_contours = [contour for contour in contours if cv2.contourArea(
contour) != 0]
305
306 coordinates_list = iterateArea(final_contours, drawn_lines, iterate=True)
307
308 # Debugging output
309 print(f"Coordinates list for {image_path}: {coordinates_list}")
310
311 if not coordinates_list:
312     print(f"No valid coordinates found for image {image_path}")
313     return
314
315 for item in coordinates_list:
316     if len(item) != 2:
317         print(f"Skipping invalid item in coordinates_list: {item}")
318         continue
319     x, y = item
320     if len(x) == 0 or len(y) == 0:
321         continue
322     feature_vector = np.zeros(len(X[0]))
323     num_stars = len(x)
324     feature_vector[:num_stars] = x
325     feature_vector[num_stars:num_stars*2] = y
326
327     expected_length = len(X[0])
328     if drawn_lines is not None:
329         flattened_lines = drawn_lines.flatten()
330     else:
331         flattened_lines = np.zeros(expected_length - num_stars*2)
332
333     feature_vector[num_stars*2:num_stars*2 + len(flattened_lines)] =
flattened_lines
334

```

```
335     feature_vector = feature_vector[:expected_length]
336
337     prediction = knn.predict([feature_vector])
338     predicted_label = labels[prediction[0]]
339     print(f"The image {os.path.basename(image_path)} is classified as: {
340         predicted_label}")
341
342 # Classify all images in the image directory
343 def classify_all_images(image_directory):
344     for filename in os.listdir(image_directory):
345         if filename.endswith('.jpg') or filename.endswith('.png'): # Add more
346             extensions if needed
347             image_path = os.path.join(image_directory, filename)
348             classify_image(image_path)
349
350 # Classify all images
351 classify_all_images(image_directory)
```

Listing 5: Evaluate Model Example

6 Exoplanets, detection methods, habitability parameters

Exoplanets are planets that exist outside our solar system, orbiting stars other than our Sun. They come in a surprising variety, ranging from giant gas balls bigger than Jupiter to rocky worlds potentially similar to Earth. Astronomers use clever methods like watching for the wobble of a star or the dimming of its light to detect these distant neighbors. With thousands already discovered and more on the way, the search for exoplanets is helping us understand how common planets are and if there might be another Earth out there somewhere!

6.1 Detection methods

Exoplanets are very faint compared to the stars they orbit, making them difficult to detect directly. Astronomers use a variety of indirect methods to find these distant worlds. Here are some of the most common methods:

1. Radial velocity method:

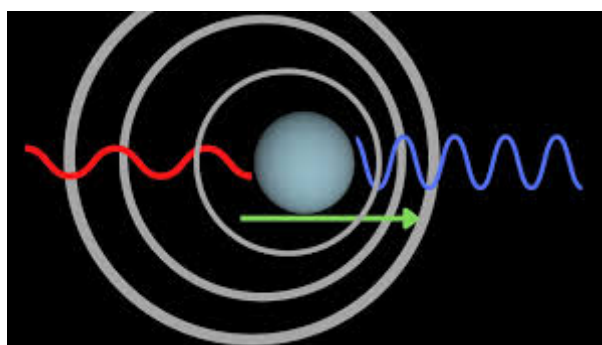
This method measures the wobble of a star caused by the gravitational pull of an orbiting planet. The star's spectrum is analyzed to detect tiny shifts in wavelength, which reveal the star's motion. This method can tell us the mass of the planet, but not its size or composition. The radial velocity method relies on the Doppler effect to detect the wobble of a star caused by an orbiting planet.

Doppler Effect:

- Light waves are stretched (redshifted) when the source is moving away from the observer and compressed (blueshifted) when moving closer.
- The amount of shift ($\Delta\lambda$) is proportional to the relative velocity (v) of the source and observer, divided by the speed of light (c):

$$\frac{\Delta\lambda}{\lambda} = \frac{v}{c}$$

- λ : Original wavelength of light
- $\Delta\lambda$: Change in wavelength



The Doppler Effect

Orbital Motion:

- The star and planet orbit a common center of mass, called the barycenter.
- As the star orbits, it experiences a slight back-and-forth motion due to the planet's pull.

- This motion causes the star's spectral lines to shift slightly in wavelength periodically.

Radial Velocity (v_{rad}):

- We can't directly measure the star's true velocity (due to the unknown orbital inclination).
- Instead, we measure the radial velocity (v_{rad}), the component of the star's velocity along our line of sight:

$$v_{\text{rad}} = v \cdot \sin(i)$$

- i : Orbital inclination (angle between our line of sight and the orbital plane)

Kepler's Third Law:

- This law relates the orbital period (P) of a planet to the semi-major axis (a) of its orbit and the total mass (M_{tot}) of the system:

$$P^2 = \frac{4\pi^2 a^3}{GM_{\text{tot}}}$$

- G : Gravitational constant

Analysis:

- By measuring the periodic shift in the star's spectral lines over time, astronomers can estimate the radial velocity (v_{rad}).
- Using Kepler's Third Law and the known mass of the star (M_*), we can then calculate the minimum mass ($M_p \sin(i)$) of the planet:

$$M_p \sin(i) = \frac{GM_* P^3}{4\pi^2 (v_{\text{rad}})^2}$$

Limitations:

- This method only provides the minimum mass of the planet, not its true mass.
- The inclination (i) is unknown, so the true planetary mass remains a mystery unless additional methods (e.g., transit) are used.
- This method is most sensitive for massive planets with short orbital periods.

2. Transit Method:

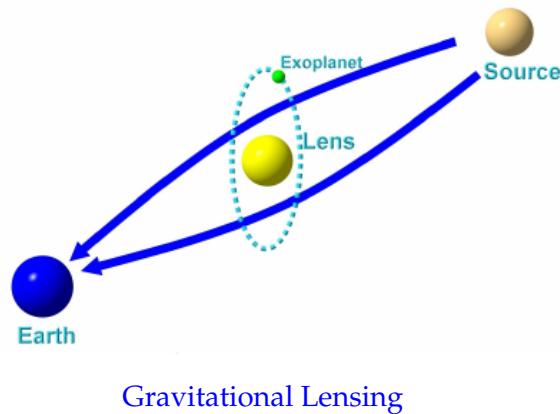
This method looks for dips in a star's brightness caused by a planet passing in front of it, as seen from Earth. The depth of the dip tells us the size of the planet relative to the star. When combined with the radial velocity method, we can determine the planet's mass and radius, and thus its density. This method can also be used to study the atmosphere of a transiting planet.

Transit Depth (D):

- This represents the fractional change in the star's brightness during the transit. It's calculated as:

$$D = \left(\frac{R_p}{R_s} \right)^2$$

- R_p : Radius of the planet



- R_s : Radius of the star

Transit Duration (T):

- This is the time it takes for the planet to completely block a portion of the star. It depends on the planet's size, orbital speed, and orbital inclination relative to our line of sight. Complex orbital mechanics are used to determine T, but it's related to the orbital period and the impact parameters.

Additional Parameters:

- By analyzing the transit light curve (brightness vs. time), we can derive other properties:
- Impact Parameter (b): This measures how close the center of the planet gets to the center of the star during transit (closer = deeper transit).
- Limb Darkening: Stars aren't perfectly uniformly bright across their surface. By modeling the limb darkening, we can get more accurate measurements of R_p .

3. Microlensing:

This method uses the gravity of a massive object, like a star or a black hole, to magnify the light of a background star. If a planet is orbiting the foreground object, it can cause a short-lived additional brightening event. This method is good for finding planets far away, but it doesn't tell us much about the planets themselves.

Microlensing relies on the gravitational lensing effect of a massive object (foreground star/-planet) to magnify the light of a background star.

Einstein Radius (R_s):

- This is the critical radius around the lensing object where light gets bent significantly. It depends on the lensing object's mass (M) and the distance to the background star:

$$R_s = \sqrt{\frac{4GM}{c^2 D_L}}$$

- G: Gravitational constant
- c: Speed of light
- D_L : Distance to the background star

Magnification (A):

- This represents the increase in brightness of the background star due to lensing. It depends on the impact parameter (b) of the planet relative to the Einstein radius:

$$A = \left(\frac{R_s}{b} \right)^2 \text{ (for small } b \text{ values)}$$

4. Astrometry:

This method measures the tiny wobble of a star caused by the gravitational pull of an orbiting planet. However, instead of measuring the star's velocity, astrometry measures its position on the sky. This method is very challenging, but it can be used to find planets in any orientation relative to our line of sight.

6.2 Planetary Habitability: The Potential for Life Beyond Earth

Planetary habitability is the study of a planet or moon's potential to support life. This complex field considers the characteristics that make an environment suitable for life as we know it, and the possibility of life existing in forms we haven't even imagined.

Key Requirements for Habitability:

- **Energy Source:** All life needs an energy source to function. On Earth, this is primarily sunlight, captured by plants through photosynthesis. However, other energy sources, like hydrothermal vents, might support life elsewhere.
- **Liquid Water:** Water is considered essential for life as we know it. It acts as a solvent for biological processes and is a key component of living cells. The presence of liquid water on a planet's surface depends on factors like temperature, atmospheric pressure, and the amount of radiation received from the host star.
- **Stable Environment:** Life thrives in relatively stable environments. Extreme fluctuations in temperature, radiation, or other factors can be detrimental.

Factors Affecting Habitability:

- **Host Star:** The characteristics of a planet's host star significantly influence its habitability. The star's luminosity, variability, and spectral class (temperature) all play a role.
- **Planetary Characteristics:** A planet's size, mass, composition, atmosphere, and geological activity all contribute to its potential for life. Larger planets can retain atmospheres better, while geological activity can provide essential nutrients and heat.
- **Orbital Characteristics:** A planet's orbit around its star also matters. The distance from the star determines the temperature on the planet's surface. A planet's tilt and rotation rate affect its climate and seasonal variations.

The Search for Habitable Worlds:

With advancements in telescopes and space exploration, scientists are actively searching for habitable exoplanets (planets outside our solar system). The discovery of thousands of exoplanets has fueled this search, with a focus on those located within the habitable zone of their star system, where liquid water could exist on the surface.

6.3 Finding Potential Habitable Planets from Transit Data

Transit data can be a powerful tool for identifying potentially habitable exoplanets.

1. Planetary Size and Orbit:

- By measuring the depth of the transit (brightness dip), we can determine the relative size of the planet compared to its star. Combined with the star's known radius, this gives us the planet's actual diameter. The transit period reveals the planet's orbital period.

2. Habitability Zone:

- Knowing the star's temperature and luminosity, astronomers can define the habitable zone, the range of orbital distances where liquid water could exist on the planet's surface. Planets with sizes and orbits placing them within the habitable zone become prime candidates for further study.

3. Atmospheric Analysis:

- During transit, some starlight filters through the planet's atmosphere. By analyzing the spectrum of this faint light, scientists can potentially identify biosignatures, molecules like water vapor or methane, that might hint at a life-supporting atmosphere.



Exoplanet Kepler 452-b

7 Conclusion and Future Work

Predicting the habitability of exoplanets and detecting constellations using advanced algorithms is an exciting frontier where astrophysics meets machine learning. Here, we summarize key insights and propose strategies to enhance the precision and efficiency of these models.

7.1 Habitability Prediction Model

Concluding Insights:

Binary classification algorithms such as Logistic Regression, SVM, Decision Trees, Random Forests, K-Nearest Neighbors, Naive Bayes, and LDA have demonstrated their potential in predicting exoplanet habitability. Each algorithm offers unique strengths suited for different aspects of the task, emphasizing the importance of meticulous model selection and tuning to achieve optimal prediction accuracy.

– Strategies for Advancement

I Enhanced Data Strategies

- (a) Feature Engineering: Develop advanced features from transit data to capture nuanced relationships.
- (b) Balancing Imbalanced Data: Employ techniques like SMOTE to address skewed class distributions.

II Advanced Modeling Approaches

- (a) Ensemble Techniques: Combine diverse algorithms to leverage their complementary strengths.
- (b) Deep Learning Exploration: Investigate neural networks for their capacity to discern intricate patterns in complex datasets.

III Refinement through Rigorous Validation

- (a) Hyperparameter Refinement: Utilize systematic approaches such as grid and random search for optimal parameter configuration.
- (b) Cross-Validation Techniques: Implement robust validation methodologies like k-fold cross-validation to ensure generalizability and mitigate overfitting.

IV Insights through Interpretation

- (a) Model Interpretability: Harness tools like SHAP to decipher model decisions and attribute importance to features.
- (b) Visual Representation: Develop intuitive visualizations to facilitate understanding of model outputs and underlying data dynamics.

V Adaptive Learning and Continuous Improvement

- (a) Dynamic Model Updating: Integrate mechanisms for online learning to adapt models in real-time with incoming data streams.
- (b) Feedback Integration: Incorporate domain expert feedback iteratively to refine model predictions and enhance performance over iterations.

VI Collaborative Endeavors for Collective Advancement

- (a) Open Data Initiatives: Foster collaborations across research institutions to aggregate diverse datasets and broaden the scope of analysis.
- (b) Interdisciplinary Synergy: Embrace synergistic approaches by integrating insights from astrophysics, machine learning, and planetary science to enrich model development and validation.

```

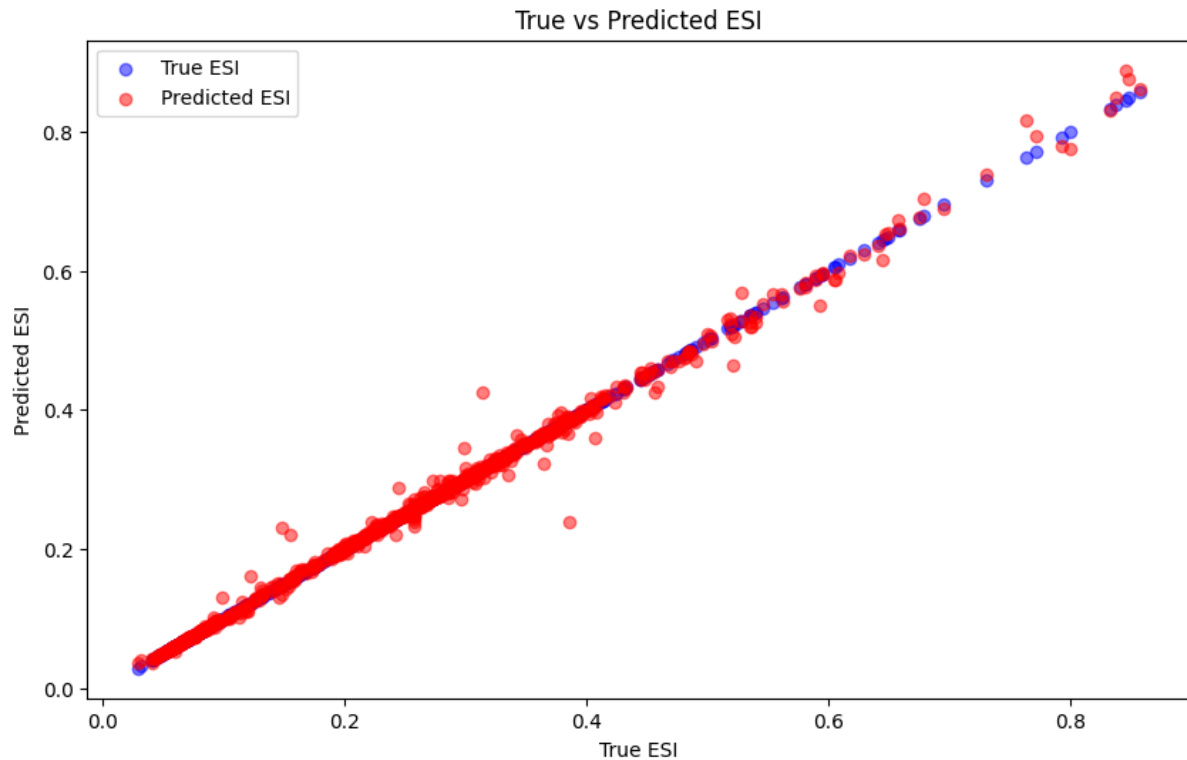
1 from sklearn.metrics import r2_score, mean_squared_error
2 import matplotlib.pyplot as plt
3 def define_features_target_split(df, target_column):
4     X = df.drop(columns=[target_column]).values
5     y = df[target_column].values
6
7     X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
8 random_state=42)
9
10    print("Training and testing data split successfully.")
11    print(f"X_train shape: {X_train.shape}, y_train shape: {y_train.shape}")
12    print(f"X_test shape: {X_test.shape}, y_test shape: {y_test.shape}")
13
14    return X_train, X_test, y_train, y_test
15 X_train, X_test, y_train, y_test = define_features_target_split(df, 'P_ESI')
16 def train_model(X_train, y_train):
17     model_pipeline = make_pipeline(StandardScaler(), RandomForestRegressor(
18 random_state=42))
19     model_pipeline.fit(X_train, y_train)
20
21     print("Model trained successfully.")
22
23     return model_pipeline
24 model_pipeline = train_model(X_train, y_train)
25
26 def evaluate_model(model_pipeline, X_test, y_test):
27     y_pred = model_pipeline.predict(X_test)
28
29     r2 = r2_score(y_test, y_pred)
30     mse = mean_squared_error(y_test, y_pred)
31
32     print('R2 Score:', r2)
33     print('Mean Squared Error:', mse)
34
35     plt.figure(figsize=(10, 6))
36     plt.scatter(y_test, y_pred, alpha=0.5)
37     plt.xlabel('True ESI')
38     plt.ylabel('Predicted ESI')
39     plt.title('True vs Predicted ESI')
40     plt.show()
41
42     return y_pred
43
44 y_pred = evaluate_model(model_pipeline, X_test, y_test)

```

Listing 6: Evaluate Model Example

R2 Score: 0.9951712314027632

Mean Squared Error: 8.329770441289886e-05



Predicted vs Actual ESI graph of our model

7.2 Constellation Detection Model

Summative Reflections:

Detecting constellations involves leveraging image processing techniques and machine learning algorithms to identify celestial patterns accurately. While methodologies like HAAR cascades and various classifiers have proven effective, continuous refinement is essential to elevate accuracy and operational efficiency.

Path to Progress

I Optimized Data Utilization

- (a) Data Quality Enhancement: Curate high-resolution, diverse datasets conducive to robust model training.
- (b) Augmentation Strategies: Employ augmentation techniques (e.g., rotation, scaling, noise addition) to augment dataset diversity and improve model resilience.

II Advanced Image Processing Paradigms

- (a) Preprocessing Excellence: Implement sophisticated preprocessing protocols to enhance image clarity, noise reduction, and feature extraction efficacy.
- (b) Feature Extraction Mastery: Leverage advanced techniques such as Histogram of Oriented Gradients (HOG) to refine feature representation and bolster model performance.

III Precision through Algorithmic Refinement

- (a) Parameter Optimization: Fine-tune algorithmic parameters to optimize performance across varying datasets and operational scenarios.

- (b) Algorithmic Diversification: Explore diverse classifiers and ensemble methodologies to discern optimal frameworks for constellation detection tasks.

IV Harnessing the Potential of Deep Learning

- (a) Incorporating Convolutional Neural Networks (CNNs): Deploy CNN architectures to capitalize on their proficiency in image feature extraction and classification.
- (b) Transfer Learning Strategies: Harness pre-trained models and adapt them through transfer learning paradigms to expedite model convergence and enhance detection accuracy.

V Operational Efficiency and Real-Time Deployment

- (a) Performance Optimization: Implement strategies to streamline computational processes, ensuring swift and efficient model execution.
- (b) Parallel Processing Capabilities: Capitalize on GPU-accelerated computing to expedite image processing tasks and bolster real-time detection capabilities.

VI Stakeholder Engagement and Iterative Enhancement

- (a) User-Centric Tools: Develop interactive tools facilitating collaborative verification and validation of model outputs by stakeholders, including astronomers and domain experts.
- (b) Continuous Feedback Integration: Establish feedback loops to iteratively refine model predictions, leveraging user insights and domain expertise to drive iterative enhancements.

VII Collaborative Innovation and Knowledge Sharing

- (a) Community Engagement: Foster a culture of collaboration and knowledge exchange among diverse stakeholders, including researchers, amateur astronomers, and technology enthusiasts.
- (b) Cross-Disciplinary Synergy: Embrace interdisciplinary collaborations to harness collective expertise and accelerate advancements in constellation detection methodologies.

References

- [1] Image processing and techniques- <https://www.v7labs.com/blog/image-processing-guide>
- [2] Cascading-Classifiers, Wikipedia

Resources

- **Our Github Repo:** <https://github.com/D3bug1t/ExoStellarNexus>
- **Our Project Website:** <http://d3bug1t.github.io/Exosite>
- **Wonderful Research Paper by Ji Liu Wang:** <https://web.stanford.edu/class/ee368>